

基于Apache Arrow与CUDA内存模式的TPC-H Q5查询实验

徐子桓 2025104082

摘要 本文围绕内存数据库中的CPU-GPU协同查询处理，实现了一个固定的TPC-H Q5物理执行器。实验保留Q5需要的六张表和字段，使用Apache Arrow IPC作为可复现的数据交换格式，并比较Arrow CPU算子、专用CPU实现、三种CUDA内存模式以及cuDF算子库。三种CUDA模式分别是显式PCIe复制、managed memory 预取和mapped pinned host memory直接访问。项目先用tiny数据检查查询逻辑，再在官方TPC-H SF1数据上验证结果并测量时间。当前实现没有做完整SQL解析和代价优化，GPU也只负责最后的lineitem扫描，因此结论只适用于本项目的固定查询和SF1规模。实验中还发现，原实现逐行截断收入会偏离官方答案，因此本文改为保留四位小数累加，最后显示时再舍入。本轮六种实现得到相同结果；在三次重复的内部计时中，8线程专用CPU为22.86 ms，三种CUDA路径为238.70至323.24 ms。冷进程墙钟时间则受到输入格式影响，Arrow IPC路径明显少于C++ 文本装载路径。

关键词 内存数据库；TPC-H Q5；Apache Arrow；CUDA；统一虚拟寻址；cuDF

Abstract This report studies CPU-GPU query processing with a fixed implementation of TPC-H Query 5. Apache Arrow IPC is used as the reproducible interchange format. The experiment compares an Arrow CPU baseline, a specialized CPU executor, three CUDA memory modes, and a cuDF operator baseline. The current prototype does not implement a general SQL optimizer, and the GPU only scans and aggregates the lineitem table. Therefore, the conclusions are limited to the implemented query and the SF1 data set.

Keywords in-memory database; TPC-H Q5; Apache Arrow; CUDA; UVA; cuDF

1 引言

GPU具有较高的并行计算能力和显存带宽，但数据库查询不只包含计算，还包含数据装载、连接准备、主机与设备之间的数据移动等步骤。只比较一个CUDA核函数容易得到不完整的结论。因此，本实验选用TPC-H Q5，把CPU列式算子、专用CPU 执行器、不同GPU内存模式和GPU算子库放在同一查询语义下进行对比。

本项目不是通用数据库。它没有SQL解析器、逻辑优化器和事务模块，而是把Q5 的连接顺序和过滤过程直接写成物理执行计划。这样实现范围较小，适合课程实验，但也意味着结果不能直接推广到其他查询。

2 查询与数据表示

Q5包含六张表。region和nation用于确定地区与国家，customer和orders用于确定客户与订单，supplier和lineitem用于供应商匹配和收入计算。查询选择指定地区及一年日期范围内的订单，

要求客户与供应商属于同一国家，最后按国家聚合

$$\text{revenue} = \sum l.\text{extendedprice} \times (1 - l.\text{discount}). \quad (1)$$

项目只保留查询需要的列。字符串使用字典编码，日期转换为从Unix epoch开始的天数。价格以分为单位保存，折扣以百分之一为单位保存。单行收入用整数表示为

$$r_{10^4} = p_{cent} \times (100 - d_{0.01}), \quad (2)$$

先按 10^4 尺度求和，输出时再舍入到两位小数。这一处理避免原版本逐行截断不足一部分的问题。

数据准备脚本把TPC-H的六个.tbl文件转换为Arrow IPC文件，同时写入schema、行数和SHA256校验信息。Arrow数据集作为PyArrow与cuDF基线的共同输入。专用C++/CUDA路径目前仍从.tbl构造连续数组，这是本实验尚未统一的地方。

3 查询执行方法

3.1 CPU过滤传播

执行器先在CPU端找到目标region中的nation，然后构造supplier、customer和order到nation的三个

直接索引数组，以下分别记为 $S[s]$ 、 $C[c]$ 和 $O[o]$ 。日期或地区条件不满足的位置写为-1。最后扫描lineitem，只要订单国家与供应商国家相同，就把收入加入对应国家。该方法避免物化六表连接，但依赖TPC-H键值比较稠密的特点。

专用CPU版本把lineitem划分为连续区间，每个线程使用本地国家聚合数组，结束后再归并。Arrow CPU版本使用PyArrow的filter、join、group by和sort算子，作为通用列式算子库对照。

3.2 CUDA内存模式

三个手写CUDA后端复用CPU生成的过滤传播数组，并执行同一个lineitem聚合核：

- gpu-copy使用device buffer和显式复制；
- gpu-managed使用统一内存并在执行前预取；
- gpu-mapped使用映射锁页内存，GPU经PCIe读取主机数据。

这三个模式比较的是输入数组到GPU的存放和访问方式，并不是三种不同的连接算法。cuDF版本则通过通用GPU DataFrame的merge、filter、groupby和sort算子执行Q5，用来观察专用核函数与算子库之间的差别。

4 实验设计

实验分为两级。tiny数据只有6条lineitem，用来检查日期、地区、同国家条件和收入公式。正式数据使用TPC-H V3.0.1 dbgen生成的SF1。每个可用后端先预热，再独立进程重复3次并保存原始CSV、环境信息和结果哈希。服务器包含2路AMD EPYC 9654和NVIDIA GeForce RTX 4090，C++路径使用8个线程，软件环境为CUDA 12.6、PyArrow 23.0.1和cuDF 26.06.00。正确性要求不同后端输出的国家顺序和 10^4 尺度收入相同，并与官方q5.out的两位小数结果一致。

为了缩短课程项目周期，正式实验只做SF1，没有继续做SF10；也没有做查询并发、NUMA绑定和完整Nsight分析。因此，本文主要回答“这些实现能否正确运行、主要时间花在哪里”，不尝试给出普遍的性能模型。

5 实验结果

5.1 正确性

tiny数据的预期结果为JAPAN 190.00、INDIA 90.00。修正decimal语义后，CPU与三种CUDA模

式在 10^4 尺度上的结果哈希一致。

SF1上六种实现的结果哈希均为542abf4003633c7c。表1列出的五行与官方q5.out逐行一致，核对过程没有把字符串转换为浮点数。

Table 1 SF1 Q5结果与官方答案核对

国家	收入
INDONESIA	55502041.17
VIETNAM	55295087.00
CHINA	53724494.26
INDIA	52035512.00
JAPAN	45410175.70

5.2 性能观察

表2给出一次预热、3次正式运行的中位数。内部total由各后端报告，冷进程墙钟由外部控制器测量。专用CPU在数据已装入内存后的查询主体最短。三种CUDA路径都没有在SF1一次性查询中抵消初始化、分配和内存管理成本。

Table 2 SF1时间中位数 (ms)

后端	内部total	冷进程墙钟
CPU (8线程)	22.859	5373.400
gpu-copy	238.698	5597.830
gpu-managed	270.005	5654.902
gpu-mapped	323.239	5717.684
PyArrow	1069.345	1279.315
cuDF	1907.409	2347.973

更细的CUDA计时能说明内存模式差异。gpu-copy的H2D中位数为6.577 ms，kernel为0.230 ms；managed的预取阶段为6.299 ms，kernel为0.216 ms；mapped的显式H2D只有0.016 ms，但kernel增加到2.703 ms。这说明mapped并不是取消PCIe访问，而是把传输从显式复制变成核函数运行期间的远程读取。

两种时间列不能混为同一种排名。C++/CUDA的内部计时在.tbl已读入内存后开始，而PyArrow/cuDF把Arrow IPC读取和对象构造包含在内部计时中；外部墙钟统一包含子进程启动和装载。Arrow墙钟较短主要反映IPC列式输入比当前C++文本解析更合适，不能单独证明通用Arrow算子的查询执行快于专用CPU。

6 讨论与不足

本实验有几处比较明显的不足。第一，专用C++路径和Arrow/cuDF路径还没有完全统一数据装载口径；前者读取文本并构造数组，后者读

取Arrow IPC。第二，GPU只做lineitem扫描，不是完整的GPU查询计划。第三，三种CUDA模式每次运行都重新分配内存，无法代表常驻数据库缓冲池。第四，只测试SF1，数据量不足以说明更大规模下的交叉点。第五，聚合使用全局原子加，没有实现block内局部归约。

这些问题不影响本次“实现并比较不同内存模式”的基本目标，但会限制性能结论。后续最直接的改进是让所有后端从同一Arrow批次装载、复用GPU缓冲区，再增加SF10和多查询实验。

7 结论

本文完成了一个可以复现的TPC-H Q5 CPU-GPU实验原型。实现表明，GPU内存模式的差别不仅是API差别，还会改变PCIe传输发生的位置和核函数的数据访问代价。对于当前一次性执行的SF1查询，专用CPU的查询主体最短，GPU固定

开销没有被六百万行lineitem摊薄；但统一墙钟又显示文本装载会反过来成为主要成本。项目同时通过官方答案发现并修复了原有decimal截断问题。由于系统仍是固定查询执行器、实验规模也较小，本文把更大规模和完整GPU查询计划留作后续工作。

- [1] Transaction Processing Performance Council. TPC Benchmark H Standard Specification Revision 3.0.1[EB/OL]. <https://www.tpc.org/tpch/>.
- [2] Apache Arrow Project. Apache Arrow Documentation[EB/OL]. <https://arrow.apache.org/docs/>.
- [3] NVIDIA. CUDA C++ Programming Guide[EB/OL]. <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>.
- [4] RAPIDS. cuDF Documentation[EB/OL]. <https://docs.rapids.ai/api/cudf/stable/>.